TikuOS Memory Management

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 11, 2026

Outline

1. Why Arena Allocation?
2. Architecture & Data Structures
3. API Reference
4. Usage Examples
5. File Organization
6. API Summary
7. Test Suite
8. Summary

The Problem with malloc/free on MCUs

Traditional heap (malloc/free):

- Per-object metadata overhead
- External fragmentation over time
- No MMU to reclaim fragments
- Non-deterministic allocation time
- Fatal on 2–8 KB SRAM

Arena (bump-pointer) allocator:

- O(1) allocation — advance a pointer
- Zero per-object metadata
- Zero fragmentation
- Bulk free by resetting the pointer
- Deterministic timing

Fragmentation is Fatal

On an MCU with 2 KB SRAM and no MMU, a fragmented heap can make 50% of memory permanently unusable.

Ideal Use Cases

Temporary buffers, packet processing, sensor data batching — any scenario where allocations share a common lifetime.

Arena Control Block

kernel/memory/tiku_mem.h

```
typedef struct {
    uint8_t *buf;
    tiku_mem_arch_size_t capacity;
    tiku_mem_arch_size_t offset;
    tiku_mem_arch_size_t peak;
    tiku_mem_arch_size_t count;
    uint8_t id;
    uint8_t active;
} tiku_arena_t;
```

Field	Purpose
`buf`	Caller-provided backing buffer
`capacity`	Total buffer size (bytes)
`offset`	Current bump pointer
`peak`	Lifetime high-water mark
`count`	Allocations since last reset
`id`	Debug identifier (0–255)
`active`	Non-zero if initialized

Key Design

The arena **does not own** the buffer — the caller provides a statically allocated array. No dynamic allocation anywhere.

Supporting Types

Error codes

```c
typedef enum {
    TIKU_MEM_OK          =  0,
    TIKU_MEM_ERR_INVALID = -1,
    TIKU_MEM_ERR_NOMEM   = -2,
    TIKU_MEM_ERR_FULL    = -3,
    TIKU_MEM_ERR_NOT_FOUND = -4
} tiku_mem_err_t;
```

Statistics snapshot

```c
typedef struct {
    tiku_mem_arch_size_t total_bytes;
    tiku_mem_arch_size_t used_bytes;
    tiku_mem_arch_size_t peak_bytes;
    tiku_mem_arch_size_t alloc_count;
} tiku_mem_stats_t;
```

arch/msp430/tiku_mem_arch.h

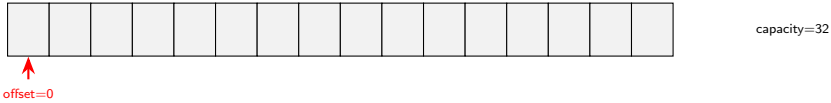
```c
/* MSP430: 16-bit arch, SRAM <= 64 KB
   uint16_t saves RAM vs uint32_t */
typedef uint16_t
    tiku_mem_arch_size_t;

/* 2-byte alignment: unaligned word
   access causes bus fault */
#define TIKU_MEM_ARCH_ALIGNMENT 2U
```

Portability

Change `ALIGNMENT` and size type to port to 32/64-bit targets.

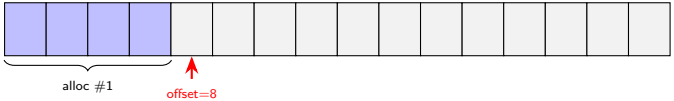
How the Bump Pointer Works

After create():

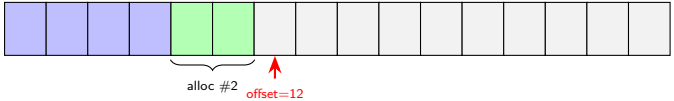


capacity=32

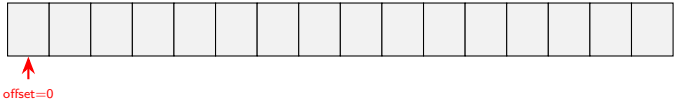
After alloc(8):



After alloc(4):



After reset():



peak=12

tiku_arena_create()

```
tiku_mem_err_t tiku_arena_create(tiku_arena_t *arena,
                                 uint8_t *buf,
                                 tiku_mem_arch_size_t size,
                                 uint8_t id);
```

- Initializes an arena over a **caller-provided** static buffer
- Sets offset, peak, count to 0; marks arena as active
- Returns `TIKU_MEM_ERR_INVALID` if `arena` or `buf` is NULL

Usage:

```
static uint8_t buffer[256];
static tiku_arena_t arena;

tiku_mem_err_t err = tiku_arena_create(&arena, buffer, sizeof(buffer), 1);
if (err != TIKU_MEM_OK) {
    /* handle error */
}
```

tiku_arena_alloc()

```c
void *tiku_arena_alloc(tiku_arena_t *arena, tiku_mem_arch_size_t size);
```

- Bump-pointer allocation — **O(1)** complexity
- Automatically aligns to `TIKU_MEM_ARCH_ALIGNMENT` (2 bytes on MSP430)
- Updates peak (high-water mark) and allocation count
- Returns `NULL` if: arena is NULL/inactive, size is 0, or arena is full

Usage:

```c
/* Allocate a sensor reading buffer */
uint16_t *readings = tiku_arena_alloc(&arena, 10 * sizeof(uint16_t));
if (readings == NULL) {
    /* arena full */
}

/* Odd sizes are rounded up: alloc(3) uses 4 bytes (2-byte alignment) */
uint8_t *pkt = tiku_arena_alloc(&arena, 3);  /* actually uses 4 bytes */
```

Alignment: align_up()

kernel/memory/tiku_mem.c (private helper)

```
static tiku_mem_arch_size_t align_up(tiku_mem_arch_size_t size)
{
    const tiku_mem_arch_size_t mask = TIKU_MEM_ARCH_ALIGNMENT - 1U;
    return (size + mask) & ~mask;
}
/* Example (alignment = 2): 1 -> 2, 2 -> 2, 3 -> 4, 5 -> 6 */
/* Example (alignment = 4): 1 -> 4, 3 -> 4, 5 -> 8, 8 -> 8 */
```

Why Alignment Matters

- MSP430 is a 16-bit architecture — unaligned word access causes a **bus fault**
- Every returned pointer must be on an even address boundary
- The formula `(size + (align-1)) & ~(align-1)` rounds up to the next multiple
- Platform-independent: change `TIKU_MEM_ARCH_ALIGNMENT` for 32/64-bit

tiku_arena_reset() vs tiku_arena_secure_reset()

Normal reset — O(1)

```c
tiku_mem_err_t
tiku_arena_reset(tiku_arena_t *arena)
{
    if (arena == NULL)
        return TIKU_MEM_ERR_INVALID;

    arena->offset = 0;
    arena->count  = 0;
    /* peak preserved */

    return TIKU_MEM_OK;
}
```

- **Does NOT zero memory**
- ~6 cycles regardless of buffer size
- Use for general-purpose reuse

Secure reset — O(n)

```c
tiku_mem_err_t
tiku_arena_secure_reset(
    tiku_arena_t *arena)
{
    if (arena == NULL)
        return TIKU_MEM_ERR_INVALID;

    tiku_mem_arch_secure_wipe(
        arena->buf, arena->capacity);

    arena->offset = 0;
    arena->count  = 0;

    return TIKU_MEM_OK;
}
```

- **Zeros entire buffer**
- ~5 cycles/byte on MSP430
- Use for crypto keys, credentials

Secure Wipe: Architecture Layer

arch/msp430/tiku_mem_arch.c

```
void tiku_mem_arch_secure_wipe(uint8_t *buf, tiku_mem_arch_size_t len)
{
    volatile uint8_t *p = (volatile uint8_t *)buf;
    tiku_mem_arch_size_t i;

    for (i = 0; i < len; i++) {
        p[i] = 0;
    }
}
```

Why volatile?

Without it, GCC -O2 sees the buffer is never read after zeroing and **elides the entire loop**. The volatile qualifier forces every store to execute.

Buffer Size	Cycles (MSP430X)	Time @ 16 MHz
64 B	~320	20 μs
256 B	~1280	80 μs
2048 B	~10240	640 μs

tiku_arena_stats()

```
tiku_mem_err_t tiku_arena_stats(const tiku_arena_t *arena,
                                tiku_mem_stats_t *stats);
```

- Returns a snapshot: total, used, peak, and allocation count
- O(1) — direct field copy
- Returns `TIKU_MEM_ERR_INVALID` if either pointer is NULL

Usage:

```
tiku_mem_stats_t stats;

tiku_arena_stats(&arena, &stats);

TIKU_PRINTF("Total: %u B\n", stats.total_bytes);
TIKU_PRINTF("Used:  %u B\n", stats.used_bytes);
TIKU_PRINTF("Peak:  %u B\n", stats.peak_bytes);   /* lifetime max */
TIKU_PRINTF("Count: %u\n",   stats.alloc_count);
```

Peak Tracking

peak_bytes records the lifetime high-water mark — it **never decreases**, even across resets. Useful for capacity planning and right-sizing buffers.

When to Use an Arena

Arena is the right choice when...

- All allocations share a **common lifetime**
- You allocate, use, and discard as a batch
- No individual free is needed
- You want O(1) alloc with zero overhead

Examples:

- Parse a sensor frame → discard
- Format a radio packet → transmit → discard
- Batch ADC readings → average → discard

Arena is **not** right when...

- Objects have independent lifetimes (packet arrives at time 1, processed at time 5)
- You need to free individual objects
- Objects are different sizes and recycled one at a time

→ Use the **pool allocator** instead.

The Key Insight

Arena = "allocate many, free all at once."
Pool = "allocate one, free one."
Together they replace `malloc`/`free`.

Example 1: Sensor Data Pipeline

```c
/* Cyclic pattern: alloc -> use -> reset -> repeat */
static uint8_t sensor_buf[128];
static tiku_arena_t sensor_arena;

void sensor_init(void) {
    tiku_arena_create(&sensor_arena, sensor_buf, sizeof(sensor_buf), 1);
}

void sensor_cycle(void) {    /* called every 100 ms by the scheduler */
    /* 1. Allocate temporary buffers for this cycle */
    uint16_t *raw = tiku_arena_alloc(&sensor_arena, 10 * sizeof(uint16_t));
    uint8_t  *pkt = tiku_arena_alloc(&sensor_arena, 24);

    if (raw == NULL || pkt == NULL) { return; }  /* arena full */

    /* 2. Use the memory -- all allocations alive at the same time */
    adc_read_batch(raw, 10);
    format_packet(pkt, raw, 10);
    radio_transmit(pkt, 24);

    /* 3. All done -- reset the arena for the next cycle [O(1)] */
    tiku_arena_reset(&sensor_arena);
    /* raw and pkt are now invalid, but we don't need them anymore */
}
```

Example 2: Capacity Planning with Stats

```c
void health_report(void) {
    tiku_mem_stats_t stats;
    tiku_arena_stats(&sensor_arena, &stats);

    TIKU_PRINTF("Arena: %u/%u bytes used (%u allocs)\n",
                stats.used_bytes, stats.total_bytes, stats.alloc_count);
    TIKU_PRINTF("Peak:  %u bytes (lifetime maximum)\n", stats.peak_bytes);
}
```

Right-Sizing Buffers

After running in the field for hours:

- `total_bytes` = 128
- `peak_bytes` = 44

Peak is only 34% of capacity → the buffer could be shrunk to 64 bytes, saving 64 bytes of precious SRAM.

Peak Never Decreases

`peak_bytes` survives resets. It records the **lifetime high-water mark** — the worst case your application ever hit. Use it to set buffer sizes with confidence.

Example Output

```
Arena:  0/128 bytes used (0 allocs)
Peak:  44 bytes (lifetime maximum)
```

Example 3: Why Secure Reset Matters on FRAM

Problem: FRAM remembers secrets

```c
static uint8_t key_buf[32];
static tiku_arena_t crypto_arena;

void authenticate(void) {
    tiku_arena_create(&crypto_arena,
        key_buf, sizeof(key_buf), 1);

    uint8_t *key = tiku_arena_alloc(
        &crypto_arena, 16);
    derive_session_key(key);
    encrypt_payload(key);

    /* WRONG: just resets the pointer */
    tiku_arena_reset(&crypto_arena);
    /* key bytes are still in FRAM! */
    /* A memory dump reveals the key */
}
```

Fix: secure reset wipes memory

```c
void authenticate(void) {
    tiku_arena_create(&crypto_arena,
        key_buf, sizeof(key_buf), 1);

    uint8_t *key = tiku_arena_alloc(
        &crypto_arena, 16);
    derive_session_key(key);
    encrypt_payload(key);

    /* RIGHT: zeros all bytes first */
    tiku_arena_secure_reset(
        &crypto_arena);
    /* key bytes are now 0x00 */
    /* safe from memory dumps */
}
```

FRAM vs SRAM

On SRAM, secrets vanish at power-off. On FRAM, they **persist across reboots**. Secure reset is essential for any arena holding

Example 4: Multi-Arena for Subsystem Isolation

```c
/* Independent arenas prevent subsystems from interfering */
static uint8_t net_buf[64], crypto_buf[32];
static tiku_arena_t net_arena, crypto_arena;

void init(void) {
    tiku_arena_create(&net_arena,    net_buf,    sizeof(net_buf),    10);
    tiku_arena_create(&crypto_arena, crypto_buf, sizeof(crypto_buf), 20);
}

void on_packet_received(void) {
    uint8_t *frame = tiku_arena_alloc(&net_arena, 48);
    parse_packet(frame, 48);
    tiku_arena_reset(&net_arena);              /* O(1) reuse */
}

void on_auth_complete(void) {
    uint8_t *key = tiku_arena_alloc(&crypto_arena, 16);
    verify_signature(key);
    tiku_arena_secure_reset(&crypto_arena);    /* wipe secrets */
}
```

Isolation Benefits

Resetting the network arena does not affect the crypto arena. Each subsystem's memory budget is fixed at compile time — no subsystem can starve another. Different security policies (normal reset vs secure

Memory Module Files

Layer	File	Role
Kernel (portable)	`kernel/memory/tiku_mem.h`	Public API, types, docs
	`kernel/memory/tiku_mem.c`	Arena logic, `align_up()`
Arch (MSP430)	`arch/msp430/tiku_mem_arch.h`	Alignment, size type
	`arch/msp430/tiku_mem_arch.c`	Secure wipe, arch init
Tests	`tests/memory/test_mem_arena.c`	9 arena test cases
	`tests/memory/test_mem_common.c`	Host stubs + test main
	`tests/tiku_test_config.h`	Per-test enable flags

Porting to a New Architecture

Only two files need to change:

1. Set `TIKU_MEM_ARCH_ALIGNMENT` (e.g., 4 for ARM Cortex-M)
2. Set `tiku_mem_arch_size_t` (e.g., `uint32_t` for 32-bit)
3. Implement `tiku_mem_arch_secure_wipe()` if needed

API at a Glance

Function	Returns	Complexity	Description
`tiku_arena_create`	`err_t`	O(1)	Init arena over static buffer
`tiku_arena_alloc`	`void*`	O(1)	Bump-pointer allocation
`tiku_arena_reset`	`err_t`	O(1)	Reset offset; preserve peak
`tiku_arena_secure_reset`	`err_t`	O(n)	Zero buffer + reset
`tiku_arena_stats`	`err_t`	O(1)	Snapshot of arena state
`tiku_mem_init`	`void`	O(1)	Module initialization

Error Handling

- NULL checks on every public function
- `alloc()` returns NULL on failure
- Others return `tiku_mem_err_t`
- No assertions — safe for production

Design Guarantees

- No dynamic allocation anywhere
- No individual free (by design)
- All pointers naturally aligned
- Peak survives all resets

Test Configuration

tests/tiku_test_config.h

```
/* Master enable */
#define TEST_ENABLE 1

/* Arena memory test flags */
#define TEST_MEM_CREATE       1
#define TEST_MEM_ALLOC        1
#define TEST_MEM_ALIGNMENT    1
#define TEST_MEM_FULL         1
#define TEST_MEM_RESET        1
#define TEST_MEM_PEAK         1
#define TEST_MEM_INVALID      1
#define TEST_MEM_SECURE_RESET 1
#define TEST_MEM_TWO_ARENAS   1
```

On-target:

```
# Enable flags in tiku_test_config.h
# then build and flash
make flash MCU=msp430fr5969
make monitor
```

Host-native (no hardware):

```
gcc -std=c99 -Wall -Wextra -I. \
  tests/memory/test_mem_common.c \
  tests/memory/test_mem_arena.c \
  kernel/memory/tiku_mem.c \
  -o test_tiku_mem \
  && ./test_tiku_mem
```

Host mode provides portable stubs for arch functions — no MSP430 toolchain required.

Test Cases Overview

#	Test Flag	What It Verifies
1	TEST_MEM_CREATE	Arena becomes active; ID set; initial stats zero
2	TEST_MEM_ALLOC	Non-NULL pointers; sequential placement; non-overlap
3	TEST_MEM_ALIGNMENT	Odd requests rounded up; all pointers aligned
4	TEST_MEM_FULL	NULL returned when arena exhausted; boundary check
5	TEST_MEM_RESET	Offset/count reset to 0; peak preserved; reallocation works
6	TEST_MEM_PEAK	Peak increases; never decreases; survives resets
7	TEST_MEM_INVALID	NULL arena/buf rejected; zero-size returns NULL
8	TEST_MEM_SECURE_RESET	Buffer zeroed; state reset; peak preserved
9	TEST_MEM_TWO_ARENAS	Independent arenas; separate stats; reset isolation

Test Deep Dive: Allocation & Alignment

test_tiku_mem.c — Allocation test

```c
uint8_t buf[64];
tiku_arena_t arena;
tiku_arena_create(&arena, buf, sizeof(buf), 1);

/* Allocate 8 bytes */
void *p1 = tiku_arena_alloc(&arena, 8);
TEST_ASSERT(p1 != NULL);
TEST_ASSERT(p1 == buf);                    /* starts at base */

/* Second allocation is sequential */
void *p2 = tiku_arena_alloc(&arena, 4);
TEST_ASSERT(p2 == buf + 8);                /* right after p1 */
TEST_ASSERT((uintptr_t)p2 % TIKU_MEM_ARCH_ALIGNMENT == 0);
```

test_tiku_mem.c — Alignment test

```c
/* Odd request: 3 bytes -> rounded to 4 (alignment=2) */
void *p1 = tiku_arena_alloc(&arena, 3);
void *p2 = tiku_arena_alloc(&arena, 1);

/* p2 starts at offset 4, not 3 */
TEST_ASSERT((uint8_t *)p2 - (uint8_t *)p1 >= 4);
TEST_ASSERT((uintptr_t)p2 % TIKU_MEM_ARCH_ALIGNMENT == 0);
```

Test Deep Dive: Peak Tracking & Secure Reset

test_tiku_mem.c — Peak tracking across resets

```
tiku_mem_stats_t stats;

/* Cycle 1: allocate 12 bytes */
tiku_arena_alloc(&arena, 12);
tiku_arena_stats(&arena, &stats);
TEST_ASSERT(stats.peak_bytes == 12);

tiku_arena_reset(&arena);

/* Cycle 2: allocate 32 bytes -> peak grows */
tiku_arena_alloc(&arena, 32);
tiku_arena_stats(&arena, &stats);
TEST_ASSERT(stats.peak_bytes == 32);

tiku_arena_reset(&arena);

/* Cycle 3: allocate only 8 -> peak stays at 32 */
tiku_arena_alloc(&arena, 8);
tiku_arena_stats(&arena, &stats);
TEST_ASSERT(stats.peak_bytes == 32);   /* never decreases */
```

test_tiku_mem.c — Secure reset

```
/* Fill with 0xAA, then secure reset */
```

Test Deep Dive: Arena Full & Two Arenas

Arena full test

```c
/* Tiny arena: 4 * alignment bytes */
uint8_t buf[4 * TIKU_MEM_ARCH_ALIGNMENT];
tiku_arena_t arena;
tiku_arena_create(&arena, buf,
    sizeof(buf), 1);

/* First alloc succeeds */
void *p = tiku_arena_alloc(&arena,
    2 * TIKU_MEM_ARCH_ALIGNMENT);
TEST_ASSERT(p != NULL);

/* Fill the rest */
p = tiku_arena_alloc(&arena,
    2 * TIKU_MEM_ARCH_ALIGNMENT);
TEST_ASSERT(p != NULL);

/* Arena exhausted -> NULL */
p = tiku_arena_alloc(&arena, 1);
TEST_ASSERT(p == NULL);
```

Two independent arenas

```c
uint8_t buf_a[32], buf_b[64];
tiku_arena_t arena_a, arena_b;

tiku_arena_create(&arena_a,
    buf_a, 32, 10);
tiku_arena_create(&arena_b,
    buf_b, 64, 20);

/* Allocate from both */
tiku_arena_alloc(&arena_a, 16);
tiku_arena_alloc(&arena_b, 32);

/* Reset A only */
tiku_arena_reset(&arena_a);

/* B is unaffected */
tiku_mem_stats_t s;
tiku_arena_stats(&arena_b, &s);
TEST_ASSERT(s.used_bytes == 32);
```

Summary

Memory Management Design

- Arena (bump-pointer) allocator
- O(1) alloc, O(1) reset
- Zero fragmentation
- Zero per-object metadata
- Static buffers only
- Platform-aware alignment
- Lifetime peak tracking
- Secure wipe for secrets

Test Coverage

- 9 comprehensive test cases
- Runs on-target or host-native
- Per-test enable flags
- Covers: create, alloc, alignment, full, reset, peak, invalid inputs, secure reset, multi-arena

FRAM Awareness

Secure reset protects against non-volatile FRAM persistence — secrets don't survive across reboots.

Thank You

TikuOS Memory Management

Arena Allocator for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
API Header:	`kernel/memory/tiku_mem.h`
Tests:	`tests/memory/test_mem_arena.c`